



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Hybrid MPI-OMP

High Performance Computing for Weather and Climate

Alitzel Macías, Eduardo Muñoz-Castro, Rainette Engbers

Project Group 5

12.09.2025

1. Introduction

The increase in computing power that has occurred in recent decades has led to improvements in, e.g., the spatial and temporal resolution of weather and climate models (e.g., Moreno, Arias, Cazorla, Pardo, Navarro, Rojo and Tapiador; 2020), or oceanographical models (e.g., Rautenbach et al.; 2021). However, tradeoffs between communication and computation require optimizing programming alongside resource development to achieve a good compromise between them. Consequently, computational schemes for parallelization are needed to enhance - as well as ensure - scalability of High Performance Computing (HPC) applications.

OpenMP and Message Passing Interface (MPI) are two of the most popular parallel languages for programming (Tembhurne and Sathe; 2014). The combination of both schemes can help improve computing times. For instance, Kaiser and Baden (2001) discussed the potential of combining MPI and OpenMP, showing reductions of around 20% of the run time going from 4 to 7 threads with IBM compiler, and cache blocking turned off. Additionally, Kaiser and Baden (2001) highlighted the potential added value of using OpenMP parallelism to facilitate overlapping MPI communication and computation for stencil-based grid program. However, the application of hybrid programming schemes for parallelization (i.e., models that perform communication both through message passing and memory access; Drosinos and Koziris; 2004) is still in the attention of the community due to, e.g., challenges associated with the identification of optimal configurations (e.g., Pješivac-Grbović et al.; 2007).

Typical weather and climate models use OpenMP (OMP) to parallelize in the vertical and MPI to decompose the horizontal domain. For instance, these parallel languages are used in the Weather Research and Forecasting (WRF), and several studies (e.g., Moreno, Arias, Cazorla, Pardo, Navarro, Rojo and Tapiador; 2020; Moreno, Arias, Cazorla, Pardo and Tapiador; 2020) have recognized that a suboptimal configuration has an impact on computational and energy costs.

To better understand the interaction between OMP and MPI, here we aimed to implement a hybrid parallelization with both of them and analyze some of the strengths and shortcomings associated with this approach. Specifically, we ask:

1. To what extent do OMP and MPI improve computational performance?
2. How sensitive is the computational cost to changes in the domain shape (i.e., $n_x \neq n_y$)?
3. How predictable is the optimal number of OMP threads and MPI ranks?

To do so, we performed different computational experiments run in the HPC Research Infrastructure Alps Santis. First, we compare the computational performance of implementing OMP and MPI. Second, we assess changes in computational performance due to changes in the domain size. Third, we quantify changes in computational cost given changes in the domain shape (i.e., $n_x \neq n_y$). Here, we hypothesize that square domains are better due to the decomposition of the domain made by the partitioner. Finally, we explore to what extent the optimum configuration (i.e., OMP-MPI division) can be a priori predicted based on the domain size.

2. Methodological approach

2.1. Hybrid MPI-OMP implementation

Here, we use OpenMP and MPI to test different OMP threads and MPI nodes configurations and assess their impacts on computation times. Table 1 compares both parallelization languages. To improve our understanding of OMP-MPI interaction, we defined different experiments using different domain sizes, thread and node configurations, shapes, and computing infrastructure (further details about the experimental setup are presented later). For these experiments, we updated the stencil-2d code to parallelize the vertical k-dimension using OpenMP and to add MPI domain decomposition with a partitioner. The number of halo points is fixed at 2 across all configurations and domain sizes. The MPI partitioner first decomposes along the j-direction and then along the i-direction, to

favor subdomains that are as square as possible. The halo exchange uses non-blocking communication: each rank pre-posts MPI_Irecv requests for its four neighbors (top, bottom, left, right), packs the top/bottom buffers and issues the corresponding MPI_Isends, and then calls MPI_Waitall to complete the top/bottom exchange before proceeding. Left/right buffers are packed and sent in a second phase, followed by a second MPI_Waitall to complete the left/right transfer. This pattern of Irecv $\rightarrow$ Isend $\rightarrow$ Waitall, should avoid deadlock and ensure data is available before unpacking.

Aspect	OpenMP (OMP)	MPI
Programming Model	Shared-memory parallel programming	Distributed-memory parallel programming
Memory Access	All threads share a common address space	Each process has its own local memory, explicit communication required
Scalability	Limited by the number of cores and memory on a single machine	Scales across multiple nodes and clusters
Communication	Implicit, via shared variables	Explicit, via message passing functions
Ease of Use	Easier to implement for small to medium parallel tasks	More complex; requires careful data distribution and communication design
Performance Overhead	Lower overhead for small to medium thread counts	Higher communication overhead, but can handle large-scale problems
Best Use Cases	Multi-core CPUs, shared-memory systems	Large-scale computing on distributed clusters or supercomputers

Table 1: Comparison between OpenMP and MPI

2.2. Experimental setup

To address our research questions, we define three computational experiments, which are mainly based on modifications of the configuration of the parallelization scheme as well as the dimension of the domain. Our experiments are described below:

1. We compare the computational performance of implementing OMP and MPI, both independently and jointly, over a square domain.
2. We assess the computational performance over five different square domains: (i) $n_x=n_y=128$, (ii) $n_x=n_y=256$, (iii) $n_x=n_y=512$, (iv) $n_x=n_y=1024$, (v) $n_x=n_y=2048$.
3. We configure and test different rectangular shapes for three domains to assess changes in the computational cost compared to square ones (i.e., $n_x = n_y$). The shape is determined by keeping the size constant ($n_x \times n_y$) while changing the ratios. We experiment with 3 different sizes and changing their shape ratios, shown in Table 2.

	1:1	2:1	1:2	4:1	1:4
Small	128x128	176x88	88x176	256x64	64x256
Medium	512x512	720x360	360x720	1024x256	256x1024
Large	2048x2048	2896x1448	1448x2896	4096x1024	1024x4096

Table 2: Experiment configurations for different domain shapes and three different sizes.

We conducted our experiments on a single node of the Santis cluster, part of the HPC Research Infrastructure Alps (see CSCS Alps <https://www.cscs.ch/computers/alps>) hosted at the Swiss National Supercomputing Centre. Each Santis node is equipped with an NVIDIA Grace-Hopper processor, featuring 4 Grace-Hopper “superchips.” Each superchip integrates 72 ARM cores and an H100 GPU with 96 GB of HBM3 memory, along with 128 GB of LPDDR5X RAM. This configuration yields a maximum of $4 \times 72 = 288$ CPU cores per node.

We will keep the vertical dimension ‘nz’ fixed to 64 levels, to study and isolate the performance effects of horizontal decomposition. Then, each OMP thread’s “chunk” is a full $i \times j$ plane at a given k level. The decomposition of the horizontal grid will be carried out with MPI. We want to explore both intra-module and inter-module communication and scaling costs. For the intra-module, we will work inside one NUMA domain, that is, we will stay inside only one Grace CPU. This will allow us to have memory locality and not have additional runtime due to communication across modules. For this case, since we are working only with one CPU, which has 72 cores, we then need to keep $n\text{ranks} \times n\text{threads} \leq 72$.

For the inter-module communication we will use all 4 module inside one single node. This will allow us to study cross-NUMA domains, which is what happens in real-life realistic large domains. Each MPI rank should still be bound within one module (to still have memory locality and no communication issues) so we will use ≤ 72 threads per rank.

For comparison, we also ran a single domain experiment on the Eiger.alps supercomputer. Its compute nodes are built with two sockets, and each socket consists of one AMD EPYC™ 7742 64-core processor, resulting in a maximum of $2 \times 64 = 128$ CPU cores per node.

2.3. Performance assessment

We utilize the performance monitoring tool `perf`, which leverages CPU hardware counters as a base to measure, monitor, and display performance data. This enables users to profile applications, identify performance bottlenecks, and comprehend system behavior. This tool is implemented in the bash wrapper ‘`perf_wrap.sh`’, which simplifies conducting this analysis as it makes it more flexible, e.g., to changes in the OMP-MPI configuration and names of output files. ‘`perf_wrap.sh`’ follows three main steps: (i) the script runs ‘`perf stat`’ with a set of relevant hardware counters (memory accesses, LLC misses, floating-point operations), capturing its output into the log; (ii) it executes ‘`perf record`’, collecting low-level performance data into the binary ‘`.dat`’ file while suppressing console noise; (iii) it calls ‘`perf report`’ on the recorded data, appending a readable performance summary to the log file.

3. Results

3.1. Numerical validation

To validate our 2D stencil implementation, we compare the initial condition and the final result after stencil application (e.g., Figure 1), showing the expected outcome from a stencil correctly applied: (i) the interior of the domain remains constant, with slight changes in the boundaries (i.e., minor smoothing or transition zone between the zone with values 1 and 0), (ii) no artifacts or oscillations are observed, and (iii) symmetry is maintained. Here, we showed only the verification for a medium domain size (512x512). Still, these results are extrapolatable to other dimensions, considering that we are evaluating whether there are possible artifacts due to errors in the implementation of the stencil. Additionally, we compared our results with reference runs developed in classes to check consistency, such as the OpenMP scaling, obtaining similar results (not shown).

3.2. Computational experiments

3.2.1. Running on one GH CPU

We first run the experiment on a medium-size domain using a single Grace Hopper CPU to observe scaling limited to communication within one chip (shared cache and memory) and to avoid

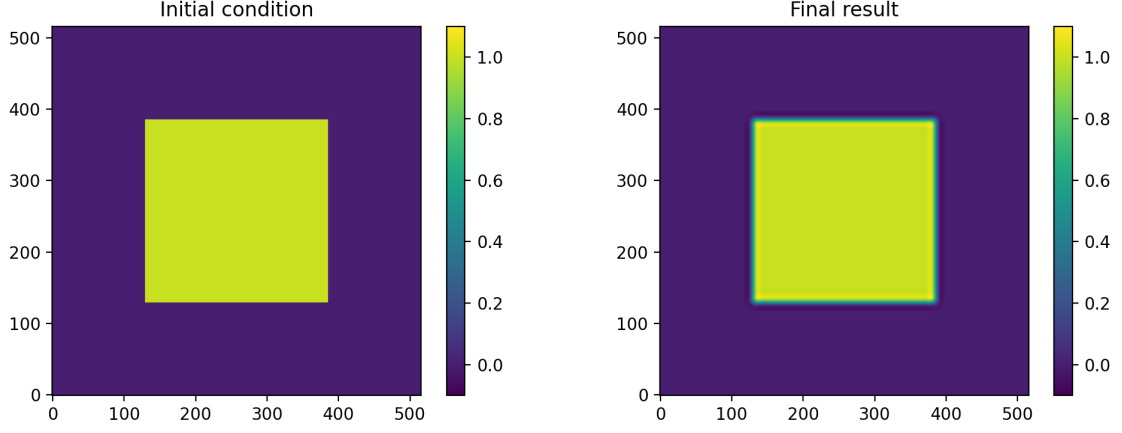


Figure 1: Example of validation of the 2D stencil update on a 512×512 domain ($n_z = 64$; iterations = 1024). Left panel: initial condition. Right panel: final condition after stencil application.

NUMA overhead from inter-socket communication. Figure 2 and Figure 3 show the speedup for the simulation on 72 cores. In Figure 2, all cores are bound to a single GH CPU within one Santis node, whereas in Figure 3 the cores are allowed to spread across all four GH CPUs on the node.

Binding all tasks to one CPU achieves substantial speedup—up to $50\times$ relative to the baseline—when using multiple MPI ranks with a single OpenMP thread per rank. Without binding, the maximum speedup is lower, and performance decreases when using two OMP threads per rank, an effect not seen in the single-CPU case. This reduction is due to the additional overhead of accessing memory across sockets. Profiling with `perf_wrap.sh` confirms that (not shown), unless explicitly bound, SLURM distributes tasks across the entire range of cores on the node (0–288), thereby introducing cross-socket memory traffic and lowering efficiency.

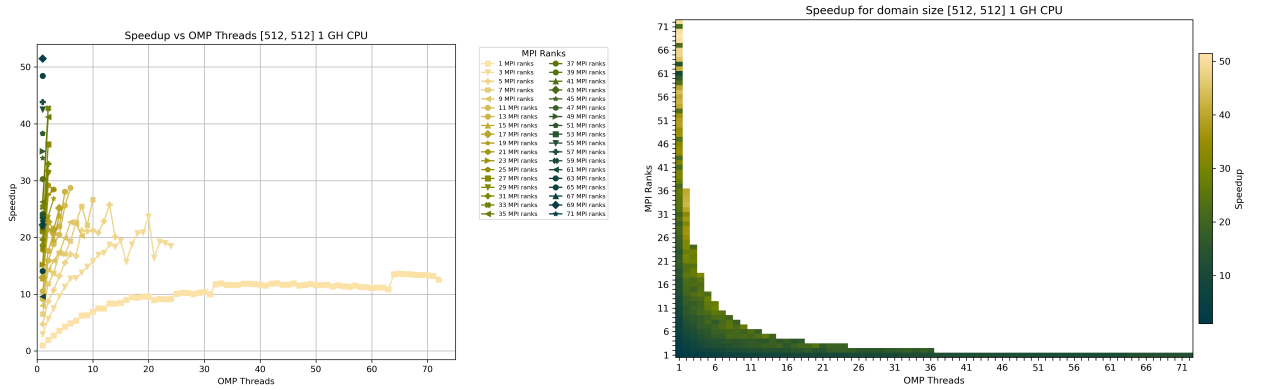


Figure 2: Speed-up for domain of size $[512, 512]$ on Santis running explicitly only on one GraceHopper CPU. Baseline runtime is ≈ 10 seconds.

3.2.2. Running on a full Santis node

Figure 4 shows runtime per gridpoint for different domain sizes and different OpenMP and MPI distributions. In all domain sizes, running with only OpenMP results in higher runtimes per gridpoint, so the slowest simulation time. Running with only MPI shows the smallest runtimes per gridpoint from the domain size $n_x=n_y=64$ onwards. In the domains $n_x=n_y=16$ and $n_x=n_y=32$, number of MPI ranks of 16 and 32 have lowest runtime per gridpoints, respectively.

The following performance analysis with `perf_wrap.sh` is done for the $[1024, 1024]$ domain size

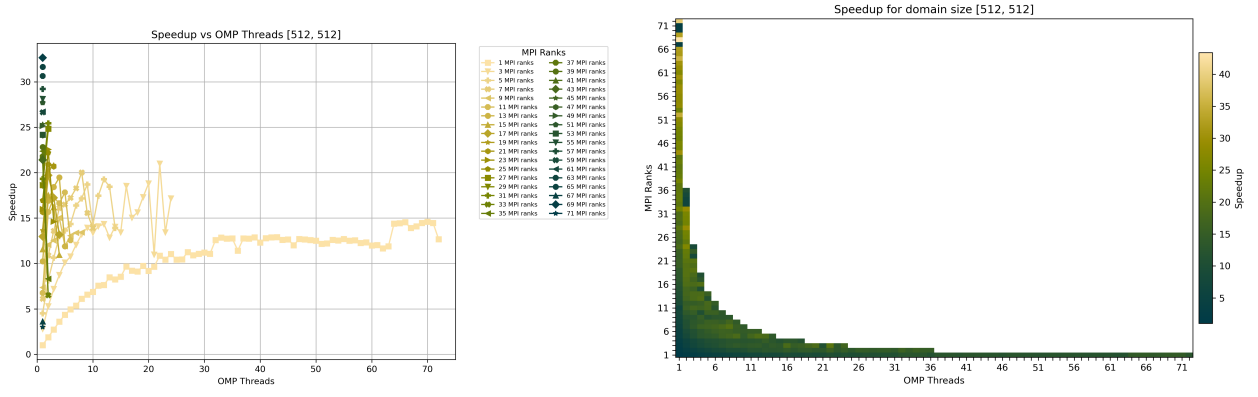


Figure 3: Speed-up for domain of size [512,512] on Santis without explicitly running on one Grace-Hopper CPU, so memory and tasks can be distributed over all 4 GH CPUs on one node. Baseline runtime is ≈ 10 seconds.

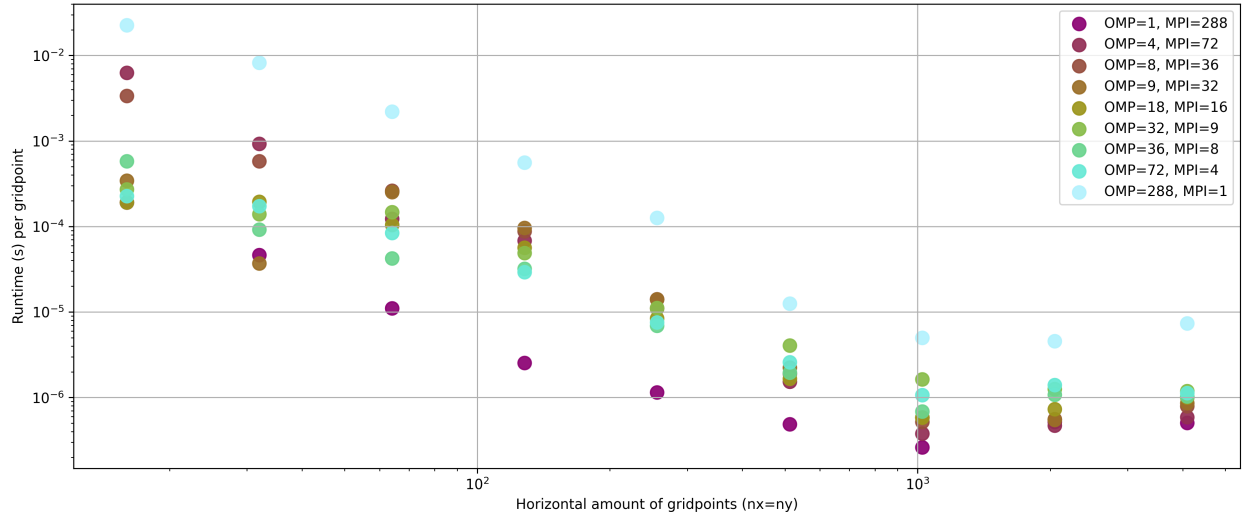


Figure 4: Runtime per gridpoint for different domain sizes (x-axis) and different MPI/OMP configurations (colors). Domain sizes range from $n_x=n_y=16$ to $n_x=n_y=4096$, with a doubling in horizontal length at each step.

on Santis. Table 3 gives the overhead in % of the 5 most computationally expensive functions for the different configurations from Figure 4. Only information of rank 0 is given, which is generally the rank which has the longest runtime. It should be noted that the percentages of time spent on different function in different ranks can be significantly different. Especially the function `MPI.Wait` can dominate in higher ranks. Here, we mostly want to show how much time is spent relatively on `apply_diffusion` in different configurations.

	OMP18_MPI16	OMP1_MPI288	OMP288_MPI1	OMP32_MPI19	OMP36_MPI18	OMP4_MPI72	OMP72_MPI14	OMP8_MPI36	OMP9_MPI32
<code>apply_diffusion.6_omp_fn.0</code>	47.67%	1.22%	76.17%	52.18%	51.00%	16.04%	49.74%	36.49%	42.11%
<code>_pmi_mmap_tmp</code>	-	95.11%	-	-	-	62.41%	-	8.27%	3.46%
<code>omp_get_num_procs</code>	39.06%	-	22.92%	29.87%	40.60%	9.41%	44.53%	31.44%	37.59%
<code>_memcpy_generic</code>	6.15%	1.80%	0.54%	5.06%	4.01%	6.38%	2.76%	14.26%	8.93%
<code>update_halo.5.constprop.1.isra.0</code>	2.36%	0.27%	-	-	1.23%	-	0.79%	2.97%	2.16%
<code>MPIR.Waitall_impl</code>	-	-	-	2.07%	-	-	-	-	-

Table 3: Top 6 functions by percentage of runtime across OMP/MPI configurations. Only the information from rank 0 is given in this table.

The fastest simulation for this domain size, using only MPI, spends very little time in the computation of `apply_diffusion` but almost the entire runtime inside `_pmi_mmap_tmp`, a MPI process management routine. This indicates that the run is dominated by MPI communication

and process coordination overhead rather than useful computation.

`_memcpy_generic` is a function responsible for moving memory. A high percentage of time in this routine suggests that the simulation is memory-bound, with performance limited by data movement rather than floating-point operations.

In contrast to the simulations with mostly MPI, simulations using OpenMP spend most of their runtime in `apply_diffusion` and `omp_get_num_procs`. The former reflects the time spent in the main computation, while the latter indicates overhead from frequent OpenMP runtime calls to query the number of threads. Together, these results show that OpenMP heavy configurations are limited by computation and OpenMP runtime overhead, whereas MPI-heavy configurations are dominated by communication and memory movement.

3.3. Communication vs. computation

To assess the interaction between communication and computation, we apply the roofline model Williams et al. (2009) to different domain sizes and OMP-MPI configurations (Figure 5). Across all domain sizes, Figure 5 shows that points are below the theoretical compute roofline, indicating that the simulations are mainly memory-bound. Small domains (e.g., 16x16 and 64x64) show low operational intensities, with performance dominated by MPI communication and memory access overheads. For medium domains (e.g., 256x256 and 512x512), performance rises, and some points are located close to the sloped memory roofline. For the largest domains here tested (i.e., 1024x1024), operational intensity increases further and performance approaches the bandwidth roofline, with some points appearing slightly above it due to potentially conservative estimates of memory traffic and effective cache utilization. It is interesting to note that the average change in performance (i.e., black marker) when moving from a 16x16 to a 64x64 domain increases by more than 400%, while the increase in performance when moving from a 256x256 domain to a 512x512 domain is only 36%. This difference becomes much smaller when moving from 512x512 to 1024x1024, where there is even a slight decrease in performance (-5%). Additionally, the variation in performance and intensity across different OMP-MPI configurations (i.e., dispersion bars associated with the black markers) diminishes as the domain size increases.

In Figure 5, we can see that the extreme OMP-MPI configurations (i.e., 1OMP-288 or 288OMP-MPI) are not the best, highlighting the need to have a good compromise between OMP-threads and MPI-nodes to achieve better efficiency. For instance, more balanced configurations such as OMP9-MPI32 or OMP36-MPI8 seem to show a more consistent performance across the different domain sizes. Additionally, it is interesting to note that some points are located outside the sloped memory roofline. This can be attributed to various factors, such as bandwidth underestimation or peak FLOPs overestimation. Furthermore, overlap of computing and memory operations, or cache effects, as well as the use of tools such as ‘perf’, could introduce some inaccuracies in performance evaluation. However, these uncertainties do not have a major impact on the findings we have reported, which indicate that (i) memory is the main bottleneck, (ii) larger domains improve performance, and (iii) a good compromise between OMP-threads and MPI-nodes leads to a better efficiency in comparison to define extreme configurations.

3.3.1. Different shape ratios

In Figure 6 we can observe the strong scaling for MPI ranks with one OpenMP thread for different shape ratios while maintaining constant the horizontal domain area. We can observe that for the ‘Small’(128²) and ‘Medium’(512²) sizes there is no clear dimensions ratio that consistently performs better than the others and that there is a lot of variability. However, for the ‘Large’(2048²) size we can observe that the best performer is the ratio ‘4:1’, followed by ‘2:1’, ‘1:1’, ‘1:2’ and lastly ‘1:4’.

In Figure 7 we can observe a similar Strong Scaling analysis, but now for the OMP threads while using only one MPI rank. Here we can observe the same pattern in which the ratios that perform the best are those that have a larger dimension x and smaller dimension y, although now we can also see this in the ‘small’ and ‘medium’ domains.

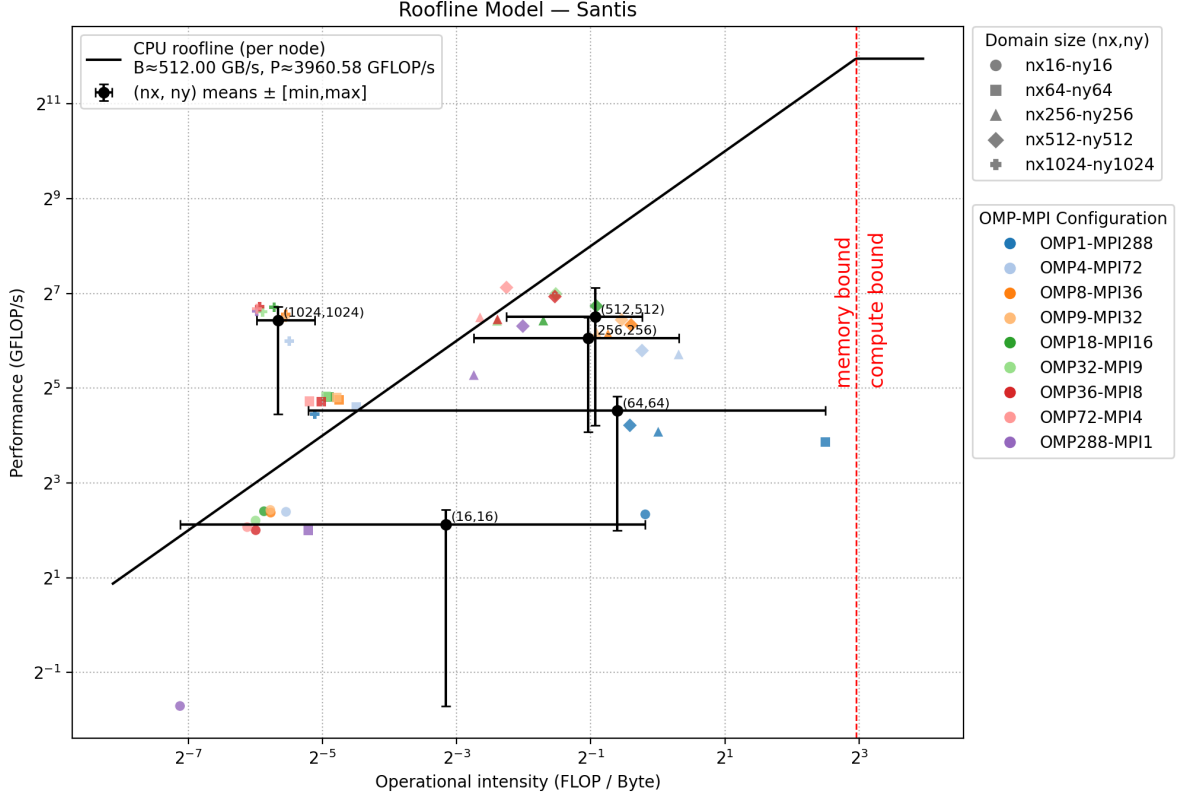


Figure 5: Roofline model of the application on Santis for different domain sizes (nx, ny) and OMP-MPI configurations. The black lines represent the roofline. Symbols indicate domain sizes (16×16 to 1024×1024), while colors distinguish OMP-MPI threads/nodes configurations. Black markers with dispersion bars show the mean $\pm$ range across runs for each domain size.

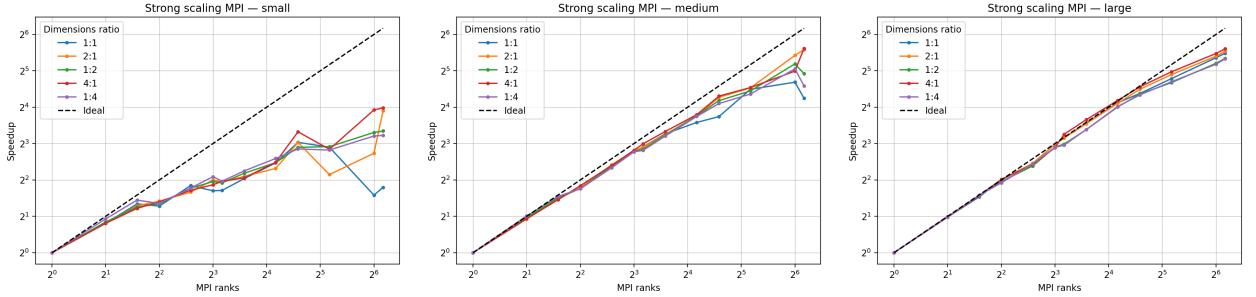


Figure 6: Strong Scaling of MPI ranks with only 1 OpenMP thread, for different shape ratios maintaining the same domain size area. The small size has 128^2 horizontal grid points, medium has 512^2 and large has 2048^2 .

Then the best ratios are those that have a larger dimension x and smaller dimension y . The MPI domain decomposition works by first dividing the y dimension and afterwards the x dimension, and an example of this can be observed in Table 4. In this table we can observe that in ratio 4:1 the x dimension is always larger than in ratio 1:4. Since Fortran has a column-major layout, then the ratios that have a longer x -dimension will have better cache hits as the i -indices (x) will be contiguous in memory, contrary to the j -indices that correspond to the y -dimension. This is why, specially for large domains, shapes that are bigger in x perform slightly better.

MPI Ranks	4:1		1:4	
	nx	ny	nx	ny
1	4096	1024	1024	4096
2	4096	512	1024	2048
3	4096	341	1024	1365
4	2048	512	512	2048
6	2048	341	512	1365
8	2048	256	512	1024
9	1365	341	341	1365
12	1365	256	341	1024
18	1365	170	341	682
24	1024	170	256	682
36	682	170	170	682
64	512	128	128	512

Table 4: MPI domain decomposition for the 'large' size for ratios '4:1' and '1:4'.

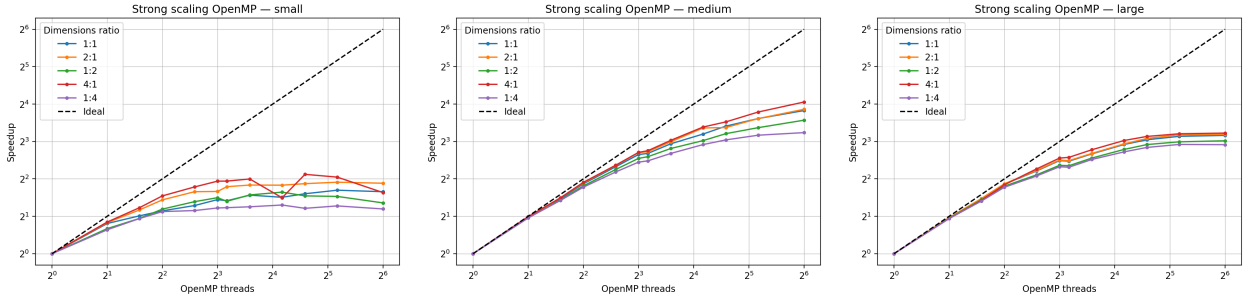


Figure 7: Strong Scaling of OMP threads with only 1 MPI rank, for different shape ratios maintaining the same domain size area. The small size has 128^2 horizontal grid points, medium has 512^2 and large has 2048^2 .

3.4. Comparison of 1 GraceHopper CPU to AMD 64-core CPU

The comparison of single-CPU experiments on Santis (NVIDIA Grace-Hopper CPU) and Eiger (AMD 64-core CPU) for the same domain size $[512, 512]$ highlights clear differences in scaling behavior (Figure 8). On Santis, the OpenMP-only experiment achieves nearly $2\times$ higher maximum speedup compared to the fastest configuration on Eiger. Additionally, on Santis, the speedup grows linearly as the number of MPI ranks increases, indicating efficient parallelization even at lower MPI counts. On Eiger, the initial increase in speedup with low MPI ranks is not linear, suggesting limited efficiency when only a few MPI ranks are active. However, as the number of MPI ranks increases further, the speedup catches up and approaches Santis levels. This difference is most likely explained by hardware characteristics, in particular, cache size and memory bandwidth. The NVIDIA GH processors provide 500 GB/s of bandwidth and 128 kB of L1 cache per core, while the AMD processors on Eiger offer roughly 200 GB/s of bandwidth and 64 kB of L1 cache per core. These architectural differences can significantly influence performance in memory-bound workloads. As MPI reduces the amount of memory contention per thread, this results in better scaling for high MPI configurations.

It should be noted that the stencil-2d optimization flags have not been adjusted when compiling in Eiger.alps compared to Santis - they have only been changed so that it manages to compile on the different hardware. Adjusting optimization flags for the specific computer architecture might lead to better scaling in Eiger.alps.

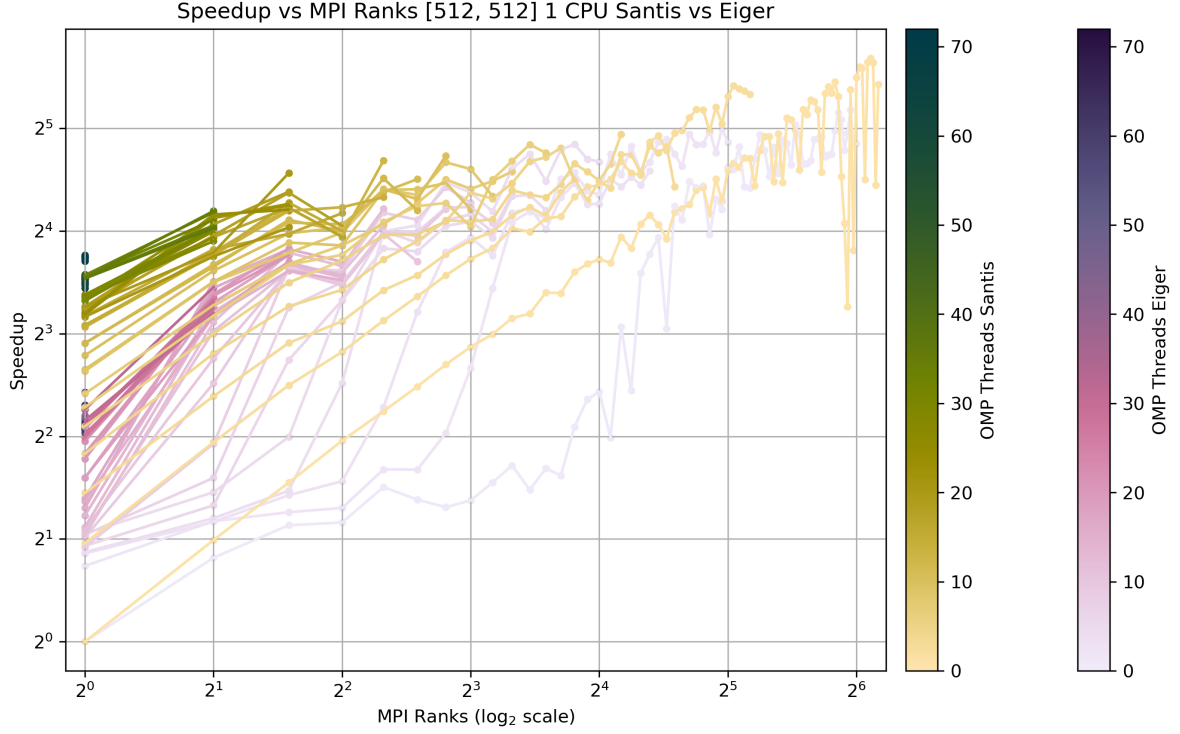


Figure 8: Comparison of speed-ups with different OMP-MPI configurations between one Nvidia GraceHopper CPU (Santis) and one AMD 64-core CPU (Eiger). Baseline runtime for Santis ≈ 10 seconds and Eiger ≈ 12 seconds.

4. Discussion

4.1. Optimal configurations

Figure 2 shows that the highest speedup for a medium domain size on a single GH CPU is achieved when using one OpenMP thread per MPI rank. Running with a single MPI rank and multiple OpenMP threads gives only limited benefits: speedup increases slightly with a few threads, but beyond ≈ 20 threads the increase flattens out. This indicates that the overhead of OpenMP (barriers, synchronization, and scheduling) becomes dominant when the workload per thread is small.

Additionally, because our stencil-2D model is memory-bound rather than compute-bound, the main optimization would come from reducing the time cost of memory access. MPI has distributed memory: each rank works on its own local subdomain stored in private memory, and only halo exchanges need to be communicated. These subdomains enforce locality. In contrast, with OpenMP's shared-memory, all threads share the same memory, leading to higher overhead and more exchange of memory in caches.

Our results show that as the domain size increases, operational intensity improves (Figure 5), potentially due to better cache reuse. At small domains, the cost of exchanging data between processes (i.e., communication) outweighs the available computational work, which can explain the wide performance variability. At medium domains, communication costs become less dominant, but memory bandwidth remains the main bottleneck, preventing performance from approaching the compute roof.

4.2. Domain decompositions

In the OpenMP scaling experiments, shapes that are longer in the x-dimension perform better than those larger in the y-dimension. This arises from Fortran's column-major layout, where the i-index (x-dimension) is stored contiguously in memory. Longer strides along this direction yield better

cache reuse and more efficient streaming. In the MPI scaling experiments, however, this advantage is only visible for large domains. For smaller domains, no aspect ratio consistently outperforms the others. While the cache-reuse argument remains valid, communication overhead between ranks becomes dominant at small sizes. Because the domain decomposition is performed first along the y-dimension, tall ratios (e.g., '1:4') are split into narrow tiles, producing small halo exchanges. In contrast, wide ratios (e.g., '4:1') generate larger tiles, so the relative communication cost is higher. These communication effects counterbalance the cache advantage of wide domains, and thus, there is no clear "best" ratio in the small-domain MPI runs.

5. Conclusion

We evaluated different OMP-MPI configurations for the decomposition of domains of different sizes and shapes. Based on our analysis, we can draw the following conclusions:

- OpenMP and MPI do not provide significant runtime improvements when domains are too small, since the overhead of thread management and MPI communication outweighs the computational cost.
- Across all domain sizes, increasing the number of MPI ranks yields the largest speedup. This is because the stencil-2D kernel is memory-bound, and MPI leverages distributed memory more effectively than OpenMP, which relies on shared memory.
- In large domains, ratios that outperform the others are those with a longer x-dimension due to Fortran's column-major layout. In smaller domains, there is no consistent outperformer ratio due to the trade-off between cache reuse and communication overhead.
- The analysis based on the roofline model shows that the studied configurations are strongly memory-bound across all domain sizes. For small domains, performance is limited by MPI communication overhead, while for larger domains, memory bandwidth dominates. Hybrid OpenMP-MPI configurations yield the best trade-off by reducing communication costs without overloading OpenMP threading, highlighting the need for careful node-level parallelization to approach hardware limits.
- Scaling efficiency depends on the supercomputer architecture, particularly cache size and memory bandwidth. Architectures with higher memory bandwidth can benefit more from OpenMP parallelization, while on systems with lower bandwidth, MPI alone can achieve comparable speedups.

To answer the question of how predictable the optimal number of OMP threads and MPI ranks is, our results suggest that it is strongly dependent on how memory-bound the model is and on the available memory bandwidth. In memory-heavy simulations or on architectures with limited bandwidth, increasing the number of MPI ranks consistently provides higher speedups compared to OpenMP. Finally, domain decomposition along the longer x-dimension takes advantage of Fortran's column-major layout, offering more predictable performance improvements and serving as a useful guideline when selecting the number of MPI ranks.

References

- Drosinos, N. and Koziris, N. (2004). Performance comparison of pure MPI vs hybrid MPI-OpenMP parallelization models on SMP clusters, *18th International Parallel and Distributed Processing Symposium, 2004. Proceedings.*, pp. 15–.
URL: <https://ieeexplore.ieee.org/document/1302919/>
- Kaiser, T. H. and Baden, S. B. (2001). Overlapping Communication and Computation with OpenMP and MPI, *Scientific Programming* **9**(2-3): 712152. .eprint:

<https://onlinelibrary.wiley.com/doi/pdf/10.1155/2001/712152>.

URL: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2001/712152>

Moreno, R., Arias, E., Cazorla, D., Pardo, J. J., Navarro, A., Rojo, T. and Tapiador, F. J. (2020). Analysis of a New MPI Process Distribution for the Weather Research and Forecasting (WRF) Model, *Scientific Programming* **2020**(1): 8148373. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2020/8148373>.

URL: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2020/8148373>

Moreno, R., Arias, E., Cazorla, D., Pardo, J. J. and Tapiador, F. J. (2020). Seeking the best Weather Research and Forecasting model performance: an empirical score approach, *The Journal of Supercomputing* **76**(12): 9629–9653.

URL: <http://link.springer.com/10.1007/s11227-020-03219-9>

Pješivac-Grbović, J., Angskun, T., Bosilca, G., Fagg, G. E., Gabriel, E. and Dongarra, J. J. (2007). Performance analysis of MPI collective operations, *Cluster Computing* **10**(2): 127–143.

URL: <https://doi.org/10.1007/s10586-007-0012-0>

Rautenbach, C., Mullarney, J. C. and Bryan, K. R. (2021). Parallel computing efficiency of SWAN 40.91, *Geoscientific Model Development* **14**(7): 4241–4247. Publisher: Copernicus GmbH.

URL: <https://gmd.copernicus.org/articles/14/4241/2021/>

Tembhurne, J. V. and Sathe, S. R. (2014). Performance evaluation of long integer multiplication using OpenMP and MPI on shared memory architecture, *2014 Seventh International Conference on Contemporary Computing (IC3)*, pp. 283–288.

URL: <https://ieeexplore.ieee.org/document/6897187/>

Williams, S., Waterman, A. and Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures, *Technical Report 1407078*.

URL: <http://www.osti.gov/servlets/purl/1407078/>